Databases I - CPIT240

LAB-6

Subqueries & General Functions

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	06/17/1987	AD_PRES	24000	-	-	90
101	Neena	Kochhar	NKOCHHAR	515.123.4568	09/21/1989	AD_VP	17000	-	100	90
102	Lex	De Haan	LDEHAAN	515.123.4569	01/13/1993	AD_VP	17000	-	100	90
103	Alexander	Hunold	AHUNOLD	590.423.4567	01/03/1990	IT_PROG	9000	-	102	60
104	Bruce	Ernst	BERNST	590.423.4568	05/21/1991	IT_PROG	6000	-	103	60
107	Diana	Lorentz	DLORENTZ	590.423.5567	02/07/1999	IT_PROG	4200	-	103	60
124	Kevin	Mourgos	KMOURGOS	650.123.5234	11/16/1999	ST_MAN	5800	-	100	50
141	Trenna	Rajs	TRAJS	650.121.8009	10/17/1995	ST_CLERK	3500	-	124	50
142	Curtis	Davies	CDAVIES	650.121.2994	01/29/1997	ST_CLERK	3100	-	124	50
143	Randall	Matos	RMATOS	650.121.2874	03/15/1998	ST_CLERK	2600	-	124	50
144	Peter	Vargas	PVARGAS	650.121.2004	07/09/1998	ST_CLERK	2500	-	124	50
149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	01/29/2000	SA_MAN	10500	.2	100	80
174	Ellen	Abel	EABEL	011.44.1644.429267	05/11/1996	SA_REP	11000	.3	149	80
176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	03/24/1998	SA_REP	8600	.2	149	80
178	Kimberely	Grant	KGRANT	011.44.1644.429263	05/24/1999	SA_REP	7000	.15	149	-

row(s) 1 - 15 of 20

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting	-	1700

row(s) 1 - 8 of 8

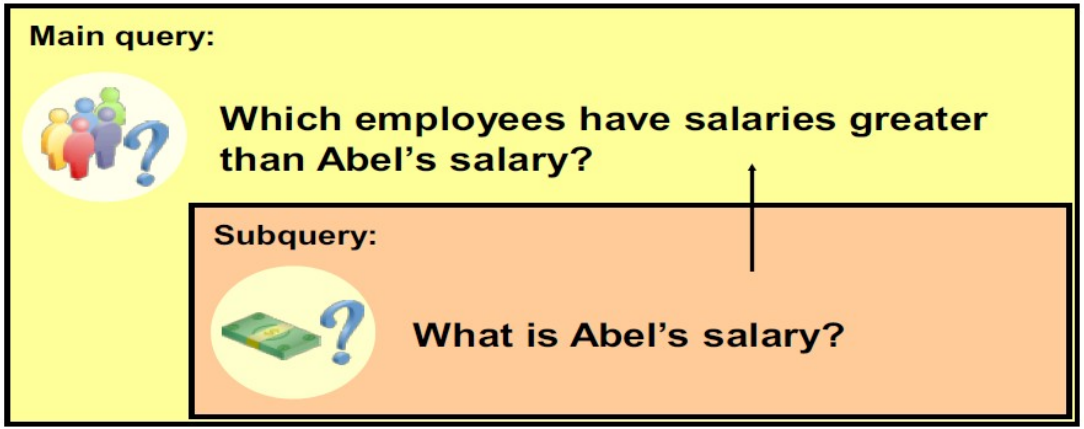
JOB_GRADES

GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
A	1000	2999
B	3000	5999
C	6000	9999
D	10000	14999
E	15000	24999
F	25000	40000

row(s) 1 - 6 of 6

Subquery

Who has a salary greater than Abel's?



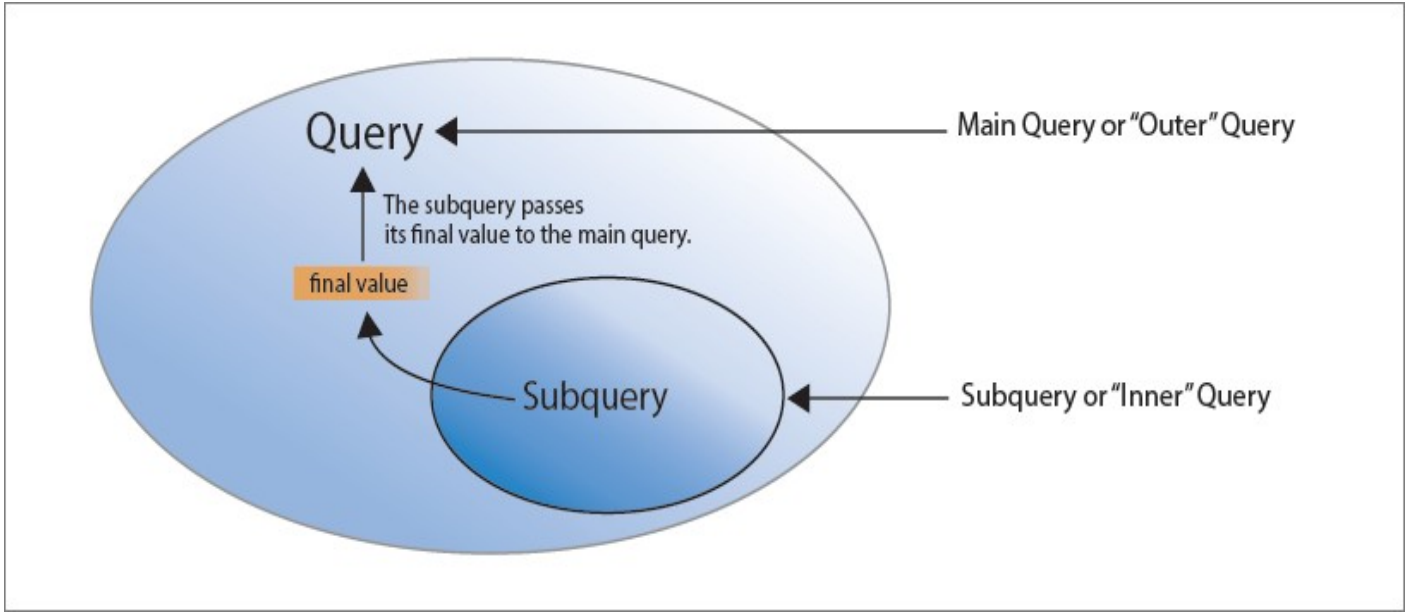
Subquery

- A subquery is a SELECT statement that is embedded in a clause of another SELECT statement.
- The subquery is often referred to as a nested SELECT, sub-SELECT, or inner SELECT statement.
- You can build powerful statements out of simple ones by using subqueries.
- They can be very useful when you need to select rows from a table with a condition that depends on the data in the table itself.

Subquery

- You can place the subquery in a number of SQL clauses, including the following:
  - WHERE clause
  - HAVING clause
  - FROM clause
- The subquery (inner query) executes once before the main query (outer query).
- The result of the subquery is used by the main query.

Subquery


Query
Main Query or "Outer" Query
The subquery passes
its final value to the main query.
final value
Subquery
Subquery or "Inner" Query

Subquery Syntax

- SQL STATEMENT

```
SELECT select_list
FROM table
WHERE expr operator
        (SELECT select_list
         FROM table);
```

- In the syntax:

operator includes a comparison condition such as >, =, or IN

NOTE: Comparison conditions fall into two classes:

  - single-row operators (>, =, >=,<, <>, <=)
  - multiple-row operators (IN, ANY, ALL).

TRY THIS

TRY THIS

1. Type the following:

```
SELECT SELECT last_name, salary
FROM   employees
WHERE  salary >
                    (SELECT salary
                    FROM   employees
                    WHERE  last_name = 'Abel');
```

2. What do you see?

LAST_NAME	SALARY
King	24000
Kochhar	17000
De Haan	17000
Hartstein	13000
Higgins	12000

SINGLE-ROW SUBQUERIES

- A single-row subquery is one that returns one row from the inner SELECT statement.
- This type of subquery uses a single row operator.

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to

TRY THIS

- The next example displays the employees whose job ID is the same as that of employee 141

TRY THIS

1. Type the following:

```
SELECT last_name, job_id
FROM   employees
WHERE  job_id =
                  (SELECT job_id
                   FROM   employees
                   WHERE  employee_id = 141);
```

2. What do you see?

```
LAST_NAME                 JOB_ID
------------------------- ----------
Rajs                      ST_CLERK
Davies                    ST_CLERK
Matos                     ST_CLERK
Vargas                    ST_CLERK
```

TRY THIS

TRY THIS

1. Type the following:

```
SELECT last_name, job_id, salary
FROM   employees
WHERE  job_id =
                (SELECT job_id
                 FROM   employees
                 WHERE  employee_id = 141)
AND    salary >
                (SELECT salary
                 FROM   employees
                 WHERE  employee_id = 143);
```

2. What do you see?

LAST_NAME	JOB_ID	SALARY
Rajs	ST_CLERK	3500
Davies	ST_CLERK	3100

CREATING TABLES FROM ANOTHER TABLE (USING SUBQUERIES)

- A method for creating a table is to apply the AS subquery clause, which both creates the table and inserts rows returned from the subquery.
- In the syntax:
  - table is the name of the table
  - Column is the name of the column, default value, and integrity constraint
  - subquery is the SELECT statement that defines the set of rows to be inserted into the new table

TRY THIS

TRY THIS

1. Type the following:

```
CREATE TABLE SALES_REP AS
        SELECT employee_id, last_name,
        salary*12 annsal, hire_date
        from employees;
```

2. You should see:

```
Table created.
```

3. Confirm that the new table has been created:

```
DESCRIBE SALES_REP
```

NULLIF

- This function is used to check if the two given expressions are equal or not.
- This function returns NULL if the given two expressions are equal.
- This function returns the first expression if the two given expressions are not equal.

TRY THIS

```
SELECT first_name, LENGTH(first_name) "expr1",
       last_name,  LENGTH(last_name)  "expr2",
       NULLIF(LENGTH(first_name), LENGTH(last_name)) result
FROM   employees;
```

FIRST_NAME	expr1	LAST_NAME	expr2	RESULT
Steven	6	King	4	6
Neena	5	Kochhar	7	5
Lex	3	De Haan	7	3
Alexander	9	Hunold	6	9
Bruce	5	Ernst	5	
Diana	5	Lorentz	7	5
Kevin	5	Mourgos	7	5
Trenna	6	Rajs	4	6
Curtis	6	Davies	6	

...

20 rows selected.

Using the COALESCE Function

- The advantage of the COALESCE function over the NVL function is that the COALESCE function can take multiple alternate values.
- If the first expression is not null, the COALESCE function returns that expression; otherwise, it does a COALESCE of the remaining expressions.
- It returns the first argument that is not null.
- The result is null only if all the arguments are null.

TRY THIS

```
SELECT last_name,
       COALESCE(manager_id,commission_pct, -1) comm
FROM   employees
ORDER BY commission_pct;
```

LAST_NAME	COMM
Grant	149
Zlotkey	100
Taylor	149
Abel	149
King	-1
Kochhar	100
De Haan	100

...

20 rows selected.

Conditional Expressions

- Provide the use of IF-THEN-ELSE logic within a SQL statement
- Use two methods:
  - CASE expression
  - DECODE function

CASE Expression

- Facilitates conditional inquiries by doing the work of an IF-THEN-ELSE statement.
- The CASE statement goes through conditions and returns a value when the first condition is met.
- Once a condition is true, it will stop reading and return the result.
- If no conditions are true, it returns the value in the ELSE clause.
- If there is no ELSE part and no conditions are true, it returns NULL.

CASE Expression

```
CASE expr WHEN comparison_expr1 THEN return_expr1
          [WHEN comparison_expr2 THEN return_expr2
           WHEN comparison_exprn THEN return_exprn
           ELSE else_expr]
END
```

TRY THIS

```
SELECT last_name, job_id, salary,
       CASE job_id WHEN 'IT_PROG'  THEN  1.10*salary
                   WHEN 'ST_CLERK' THEN  1.15*salary
                   WHEN 'SA_REP'   THEN  1.20*salary
       ELSE       salary END     "REVISED_SALARY"
FROM   employees;
```

LAST_NAME	JOB_ID	SALARY	REVISED_SALARY
...			
Lorentz	IT_PROG	4200	4620
Mourgos	ST_MAN	5800	5800
Rajs	ST_CLERK	3500	4025
...			
Gietz	AC_ACCOUNT	8300	8300

20 rows selected.

DECODE Function

- Facilitates conditional inquiries by doing the work of an IF-THEN-ELSE statement.

```
DECODE(col|expression, search1, result1
                       [, search2, result2,...,]
                       [, default])
```

In the syntax:

- expression : the value to compare.
- search: The value that is compared against expression.
- result: The value returned, if expression is equal to search.
- default: Optional. If no matches are found, the DECODE function will return default. If default is omitted, then the DECODE function will return NULL (if no matches are found).

TRY THIS

```
SELECT last_name, job_id, salary,
       DECODE(job_id, 'IT_PROG',  1.10*salary,
                      'ST_CLERK', 1.15*salary,
                      'SA_REP',   1.20*salary,
              salary)
       REVISED_SALARY
FROM   employees;
```

LAST_NAME	JOB_ID	SALARY	REVISED_SALARY
...			
Lorentz	IT_PROG	4200	4620
Mourgos	ST_MAN	5800	5800
Rajs	ST_CLERK	3500	4025
...			
Gietz	AC_ACCOUNT	8300	8300

20 rows selected.

TRY THIS

- Display the applicable tax rate for each employee in department 80

```
SELECT last_name, salary,
       DECODE (TRUNC(salary/2000, 0),
                      0, 0.00,
                      1, 0.09,
                      2, 0.20,
                      3, 0.30,
                      4, 0.40,
                      5, 0.42,
                      6, 0.44,
                         0.45) TAX_RATE
FROM   employees
WHERE  department_id = 80;
```